

# GReinSS Tutorial — Speaker Notes

*NCI Spring School on Algorithmic Cancer Biology*

One block per slide. → NOTEBOOK slides are the live-demo hand-offs.

---

## 1. GReinSS

GReinSS tutorial — NCI Spring School on Algorithmic Cancer Biology Speaker notes are in HTML comments like this one. Live-demo hand-offs are marked “→ NOTEBOOK”.

Hi everyone. Today: a hands-on tutorial on GReinSS — a method for a problem that shows up all over algorithmic cancer biology: inferring hidden combinatorial states from noisy, indirect measurements. We’ll cover the idea, the one theorem that makes it work, and then train it live on your laptop. Goal: you leave able to apply it to your own problem.

## 2. A recurring statistical inference problem in computational biology

The unifying pattern: a hidden discrete structure  $S$ , indirect observation  $X$ , and a KNOWN or partially-known likelihood  $\Pr(X|S)$ . Trees, CNA sets, isoforms — all fit. This is “self-supervised”: the physics/biology of measurement is known; the state is not.

## 3. A learning and an inference problem

Panel a: hidden distribution over states  $S^*$ , each emits an  $X$ . We model  $\Pr(S|\theta)$ . Two problems: (1) LEARN  $\theta$  from all observations jointly — the shared model couples them; (2) INFER the best state per observation. Everything today serves these two.

## 4. Why the usual tools struggle

Grouped into four families so each method fits a bucket: exact inference, variational, search, RL. EXACT / EM: the E-step needs an exact expectation over all states — tractable ONLY for special structure like an HMM chain (forward-backward); it blows up the moment  $S$  is combinatorial. HMMs are the poster child for “where these tools work” — GReinSS is for everything past that. VARIATIONAL — VI: optimizes a lower bound (ELBO) instead of the true likelihood, and you must hand-design a tractable approximate posterior  $q(S)$  over a combinatorial space — exactly what’s hard. VAE: great generative models, but the latent lives in a made-up  $\mathbb{R}^d$ , not your isoform space. SEARCH — local search: per-observation, no sharing of statistical strength across observations. RL — naive PG / GFlowNet are the closest cousins to what we do — and we’ll

see exactly why they fail. Punchline: all four families miss the SAME target — directly maximizing the joint data likelihood.

## 5. Primer on reinforcement learning (RL)

The generic RL mental model, deliberately provider-neutral and episodic: the policy builds an object action-by-action (a trajectory), a scalar reward scores the finished trajectory, and the goal is to maximize EXPECTED reward. Trace the three arrows aloud: sample  $\tau$  from the policy  $\rightarrow$  score it  $\rightarrow$  policy-gradient nudge, then repeat. The one identity they must take away is REINFORCE: you can differentiate an expectation over samples by weighting each trajectory's log-prob gradient by its reward — no gradient through the reward itself. Everything on the next two content slides is this loop with a specific reward plugged in. Contrast up top with supervised learning to anchor the audience.

## 6. GReinSS: Generative Reinforcement Learning of Structured States

**Key question:** Can we adapt reward function  $r(\tau)$  to optimize data likelihood  $\Pr(X_{1:N} \mid \theta)$ ?

Same diagram, re-labeled — say it out loud: “nothing about the machinery changes.” Actions grow a discrete structure; the trajectory's terminal state IS the object we care about  $S(\tau)$ ; the policy is a neural net; the reward (orange, highlighted) is the only novel piece and the objective is now the data log-likelihood, not a hand-picked reward. This is the pivot: GReinSS = policy gradient where the reward is engineered so that maximizing expected reward provably equals maximum-likelihood learning. Hold the suspense on the exact reward formula — that's the very next slide (the denominator is the whole trick).

## 7. Dynamically-changing rewards

The method in one line: what we optimize (the log-likelihood gradient) equals how we optimize it (a policy gradient with the dynamically rescaled reward). The numerator  $\Pr(X_i \mid \tau)$  is “how well does this trajectory explain observation  $i$ ”; the DENOMINATOR  $\Pr(X_i \mid \theta)$  is the model's total probability of  $X_i$ , which rescales each observation's contribution. Theorem 1: this policy gradient is unbiased for the log-likelihood gradient — gradient taken ONLY through  $\log \Pr(\tau \mid \theta)$ , the reward treated as constant each step.

## 8. GReinSS training loop

The training loop IS the RL cycle from the primer, with our reward plugged in: sample  $\tau$  from the policy  $\rightarrow$  score with  $\Pr(X_i \mid \tau)$   $\rightarrow$  policy-gradient update  $\theta$ . The one addition over vanilla RL is on the loop-back leg: the denominator  $\Pr(X_i \mid \theta)$  shifts as  $\theta$  learns, so each iteration we re-estimate it by sampling (average  $\Pr(X_i \mid \tau)$  over sampled trajectories). As a state gets covered its denominator

grows and its reward shrinks — automatic load balancing. API surface: the user supplies only (a) a generator for  $S$  and (b) the likelihood  $\Pr(X|S)$ . Everything else — reward machinery, sampling, gradient — is provided. That's exactly what the notebook will show: write those two functions and call `train()`.

## 9. Intuition behind dynamic rewards — why the denominator $\Pr(X_{\text{imid}}|\theta)$ ?

Policy  $\theta \equiv \Pr(\tau | \theta)$  over  $\tau_1, \tau_2, \tau_3$  ( $\tau_j$  builds  $S_j$ ); marginal  $\Pr(X_i | \theta) = \sum_{\tau} \Pr(\tau | \theta) \Pr(X_i | \tau)$ .

Reward **shrinks as it succeeds** □ the policy covers every observation.

$\theta$  IS the policy: the bars plot  $\Pr(\tau|\theta)$ , and each panel is the DIFFERENT  $\theta^*$  that its reward selects. The values are exact optima, not eyeballed. Assume one  $X_1$  and one  $X_2$ . LEFT (raw reward =  $\Pr(X_i|\tau)$ ): per-trajectory reward (.5,.3,.2); maximizing  $E_{\tau}[r]$  is linear in the policy, so all mass goes to the top,  $\tau_1 \rightarrow \theta=(1,0,0)$ . Then  $\Pr(X_2|\theta)=0$  and the joint  $L=0$ . RIGHT (rescaled): Thm 1 makes this maximize the data likelihood  $L = \Pr(X_1|\theta) \cdot \Pr(X_2|\theta) = (.5 p_1)(.3 p_2 + .2 p_3)$ .  $\tau_3$  is dominated by  $\tau_2$  for  $X_2$  (.2<.3) so  $p_3=0$ ; then  $L = .15 p_1 p_2$  with  $p_1+p_2=1$ , maximized at  $p_1=p_2=.5 \rightarrow \theta=(.5,.5,0)$ ,  $L=.25 \times .15=.0375$  (the global optimum). Punchline: the denominator = automatic load-balancing across observations.

## 10. → NOTEBOOK · Demo 1: Set reconstruction

SWITCH TO JUPYTER. Walk through: load observations → define  $\Pr(X|S)$  (one line) → build generator net → train ~200 epochs live (watch the likelihood curve rise) → infer states → compare to naive thresholding. The punchline: GReinSS denoises using structure shared across observations, beating per-pixel rounding.

## 11. Off-policy learning (when on-policy is too slow)

Practical must-have for hard problems. Instead of blindly sampling from the policy, we tilt sampling toward states that actually fit each observation — provably the best proposal. In our biology applications this is where domain knowledge enters: a fast classical method proposes candidate states, and GReinSS refines the distribution over them. Keep this slide brief unless the audience asks.

## 12. The scoreboard — who actually maximizes the likelihood?

The positive flip of the opening “why tools struggle” table — SAME four families (RL, variational, EM, search), now scored on the one question that matters. Same objective across the board; only GReinSS provably maximizes the joint

data likelihood. Naive PG collapses to one state; variational / EM-based generative models approximate via a bound or point estimates; local search has no shared model. GFlowNets are the subtle one — spell out the contrast: a GFlowNet is TRAINED to sample states with probability proportional to a KNOWN, FIXED reward  $R$  (its whole point is diverse, reward-proportional candidates). We are not matching a fixed reward at all — there is no given reward. Our “reward” is defined by the data and RESCALED every iteration (Thm 1) so that the policy-gradient objective coincides with the marginal data log-likelihood. Same REINFORCE machinery, fundamentally different target: distribution-matching a fixed reward vs. maximum-likelihood learning of a latent-variable model.

### 13. Results — simulations

Two combinatorial state types, same method. Left: graphs — GReinSS dominates especially when observations are information-poor (few walks). Right: sets — GReinSS is the only method that scales to large universes. GEM-based methods (VAE/autoregressive/diffusion) plateau; the closest RL cousins (naive PG, GFlowNet) fail. The reward rescaling is the difference.

### 14. → NOTEBOOK · Demo 2: Graph inference (pre-trained)

SWITCH TO JUPYTER (second section). Heavier model, so we ship a pre-trained checkpoint. Show: load model → simpleInference → compare predicted adjacency to the ground-truth graph we saved during pre-training → report F1 and visualize one graph. This mirrors the paper’s Fig on graph inference but on your own generated instance with known truth.

### 15. Application — RNA isoforms beat RSEM

The payoff for this audience. Isoform quantification is a textbook latent-variable problem: short reads are indirect observations of full-length transcripts. RSEM is the standard EM tool GTeX ships. Dropping GReinSS in — with a trivial  $\Pr(X|S)$  — matches long-read ground truth far better. Panel c: on MBD2, GReinSS recovers the two true isoforms with near-correct proportions; RSEM splits mass across wrong isoforms. Panel d: distribution of (GReinSS - RSEM) error is shifted negative → GReinSS wins across the genome.

### 16. GReinSS already powers two cancer methods

GReinSS isn’t just a new paper method — it’s the generalization of machinery that already produced two cancer-genomics tools. If you work on trees, CNAs, isoforms, or any grow-able discrete structure with a known likelihood, this framework likely applies to you.

### **17. When should you reach for GReinSS?**

Decision guide. The two hard requirements: an incremental generator and a likelihood. If you have those, the four-line recipe is all you need to start — exactly what the notebook demonstrates.

### **18. Thank you — let's build**

Wrap up: the method is one reward formula with a clean theorem, it beats the standard tools on simulations and on real isoform data, and it's a drop-in for discrete latent-state problems in cancer genomics. Open the notebook and try it on your own  $\Pr(X|S)$ .